Catch falling game mechanics review

10/11/2026 21:25:56
https://claude.ai/chat/b28bc265-113c-42bb-86f2-11e99763c76d

https://github.com/Abhijithmns/07_catch_falling First, read all files and…

All files read, nothing changed. (The web fetch was blocked, so I cloned…

can you identify the bugs

Bugs (Task 1, the deliberate ones) 1. **No height check in…

can you fix those bugs one task at a tie

Task 1 is fixed, and I checked it with a headless test run. These changes…

give the changed code after you fix from now on

Task 2 is done and tested headless. Still only in my clone, nothing pushe…

give the fixes for task 1 first

Task 1 fixes, both parts: **1. `game/collision.py`: no height check**…

now complete task2

Task 2 is already implemented from earlier, so there's nothing left to add…

def handle_input(self, keys_pressed): if self.game_over: return if…

Replace it with this: ```python def handle_input(self, keys_pressed): if…

now do task 3 but give the entire code with the changed ones insted

Task 3 is done and tested headless. It's only in my clone, nothing pushed…

now task 4

Task 4 is done and tested headless. It's only in my clone, nothing pushed…

https://github.com/Abhijithmns/07_catch_falling
First, read all files and summarize in under 10 lines how the game loop, spawning, collision, and basket movement currently work. Don't change anything yet

10/11/2026, 5:44:45 PM

Reviewing a project's source files to inspect the code.

- GitHub - Abhijithmns/07_catch_falling (github.com)
- Reviewing a project's source files to inspect the code.
- https://github.com/Abhijithmns/07_catch_falling/tree/main/catch-falling
- Clone repo to read source files
- Read all source files
- Done

All files read, nothing changed. (The web fetch was blocked, so I cloned the repo.)

- **Loop (`main.py`):** 60 FPS. Each frame: poll events (QUIT, KEYDOWN → `handle_keydown`), read `get_pressed()` → `handle_input`, then `update()`, `draw()`, `flip()`, `clock.tick(60)`.
- **Basket:** `handle_input` moves `basket.x` by `speed` (5 px/frame) while Left/Right is held. It clamps x to `[0, WIDTH]`, which ignores half the basket width, so it can hang half off-screen. `Basket` has unused boost fields (`boosted_frames`, `normal_speed`, `boost_speed`).
- **Spawning:** `update()` decrements `frames_until_spawn`. At ≤0 it spawns one object at `random.randint(20, WIDTH-20)`, `y=-14`, speed 3, and resets the timer to a fixed 50 frames. There is no cap and no spot-repeat avoidance.
- **Movement:** every object's `y += speed` each frame.
- **Collision:** `is_caught` checks only `basket_rect.left <= obj.x <= basket_rect.right`. There is no y check, so an object can score the moment it spawns at the top.
- **Catch loop:** iterates `self.objects` and calls `.remove(obj)` inside it, which skips the next element after each removal.

- **Misses:** objects with `y - radius > HEIGHT` are filtered out and counted as misses. At 5 misses, `game_over` is set and `update`/`input` stop. `R` re-inits the engine.
- **Rendering:** `renderer.draw_scene` draws the background, circles, and basket rect, then score and misses text. A banner shows on game over. There is no boost UI.

10/11/2026, 5:45:00 PM

can you identify the bugs

10/11/2026, 5:45:27 PM

Bugs (Task 1, the deliberate ones)

1. **No height check in `collision.py:7`.** `is_caught` only compares x against the basket's left/right edges. An object scores the instant it spawns at `y=-14` if the basket is under its x. It should also require the object's y to overlap the basket's vertical range, ideally using `basket_rect.colliderect` or a circle-vs-rect test with `obj.radius`.

2. **List mutation while iterating in `game_engine.py:65-68`.** `self.objects.remove(obj)` inside `for obj in self.objects` shifts the list, so the element after each removed one is skipped that frame. Fix by iterating over a copy, or by rebuilding the list with a comprehension, as the miss filter already does.

Related defects (Tasks 2-4)

3. **Boundary clamp ignores basket width (`game_engine.py:46`).** `max(0, min(WIDTH, x))` clamps the center, so half the basket goes off-screen. It should clamp to `[width/2, WIDTH - width/2]`.

4. **Basket logic lives in the engine.** Movement and clamping are in `GameEngine.handle_input`, not `Basket`. This isn't a bug, but it makes the boost harder to add cleanly.

5. **Spawn weaknesses (`game_engine.py:32-34, 59`).** The interval is fixed at 50 frames. x is a plain random with no minimum distance from the last spawn. There is no on-screen cap. The `20` margin is hardcoded rather than derived from the object's radius.

6. **Boost fields are dead code (`basket.py:15-17`).** `boosted_frames`, `normal_speed` and `boost_speed` exist but nothing uses them. `boost_speed == speed`, so

activating a boost would currently do nothing. There is also no key binding, timer, or HUD indicator.

Minor

- `__init__()` is called again for restart (`game_engine.py:50`), which works but is fragile.
- A missed object that overlaps the basket's x can't be double-counted, since catches are removed before the miss filter runs. Only the skip bug affects scoring.

I can fix them in task order whenever you say.

10/11/2026, 5:45:32 PM

can you fix those bugs one task at a tie

10/11/2026, 5:45:53 PM

Fixing collision detection between objects and the basket.

- Fixing collision detection between objects and the basket.
- Apply Task 1 fixes and test headless
- Install pygame and rerun tests
- Done

Task 1 is fixed, and I checked it with a headless test run. These changes are in my cloned copy only. I haven't pushed anything to your repo.

`game/collision.py`
`is_caught` now requires all three of these:

- The object's x is within the basket's left and right edges.
- The object's bottom edge has reached the basket's top (`y + radius >= top`).
- The object's center hasn't fallen past the basket's bottom (`y <= bottom`). This stops objects that slip past the basket from scoring.

`game/game_engine.py`
The catch loop no longer removes items from the list it is iterating over. It splits objects into caught and remaining in one pass, scores the caught ones, and replaces `self.objects` with the remaining list.

Test results

Case	Result
Object at spawn height (y=-14) over the basket	not caught
Object at basket height	caught
Object past the basket	not caught
Object beside the basket	not caught

Case	Result
Three adjacent objects at basket height	all 3 scored, 0 left

Say the word and I'll move on to Task 2 (basket boundaries and smooth movement).

10/11/2026, 5:46:12 PM

give the changed code after you fix from now on

10/11/2026, 5:46:37 PM

Checking whether to save a standing code-display preference.

- Checking whether to save a standing code-display preference.
- Updating memory
- Adding to memory
- Apply Task 2 fix, test, show diff
- Done

Task 2 is done and tested headless. Still only in my clone, nothing pushed. From now on I'll show the changed code after each fix.

`game/basket.py` (new method, added above `get_rect`)

```
def move(self, direction, bounds_width):
    """Move by `direction` (-1 left, 0 none, +1 right) at the
basket's
    speed, keeping the whole basket inside [0, bounds_width]."""
    self.x += direction * self.speed
    half = self.width / 2
    self.x = max(half, min(bounds_width - half, self.x))
```

`game/game_engine.py` (`handle_input`)

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return
    # Opposite keys held together cancel out (direction = 0).
    direction = int(keys_pressed[pygame.K_RIGHT]) -
int(keys_pressed[pygame.K_LEFT])
    self.basket.move(direction, WIDTH)
```

Task 1 code, which I hadn't shown before

`game/collision.py`

```
def is_caught(basket_rect, obj):
    in_x = basket_rect.left <= obj.x <= basket_rect.right
    reached_basket = obj.y + obj.radius >= basket_rect.top
    not_past_basket = obj.y <= basket_rect.bottom
    return in_x and reached_basket and not_past_basket
```

`game/game_engine.py` (catch loop in `update`)

```
remaining = []
for obj in self.objects:
    if is_caught(basket_rect, obj):
        self.score += 1
    else:
        remaining.append(obj)
self.objects = remaining
```

Task 2 tests

- Holding Left for 500 frames leaves the basket's left edge at 0.
- Holding Right for 500 frames leaves its right edge at 700.
- Pressing both keys at once leaves the basket where it is.

Movement stays smooth because it still updates every frame while a key is held. The clamp uses the basket's half-width, so it never hangs off an edge.

Task 3 (spawning) is next.

10/11/2026, 5:46:55 PM

give the fixes for task 1 first

10/11/2026, 5:48:08 PM

Task 1 fixes, both parts:

1. `game/collision.py`: no height check

```
def is_caught(basket_rect, obj):
    """Caught only when the object is horizontally over the basket
AND has
    actually reached basket height."""
    in_x = basket_rect.left <= obj.x <= basket_rect.right
    reached_basket = obj.y + obj.radius >= basket_rect.top
    not_past_basket = obj.y <= basket_rect.bottom
    return in_x and reached_basket and not_past_basket
```

2. `game/game_engine.py`: list mutated while iterating (in `update`)

Replace:

```
for obj in self.objects:                    # BUG: mutating this list
while iterating over it
    if is_caught(basket_rect, obj):
        self.score += 1
        self.objects.remove(obj)
```

with:

```
remaining = []
for obj in self.objects:
    if is_caught(basket_rect, obj):
        self.score += 1
    else:
```

```
            remaining.append(obj)
    self.objects = remaining
```

Verified headless:

- An object at spawn height over the basket is not caught.
- An object at basket height is caught.
- An object that has passed the basket or is beside it is not caught.
- Three adjacent objects at basket height all score.

Want me to start Task 3 now?

10/11/2026, 5:48:11 PM

now complete task2

10/11/2026, 5:58:53 PM

Ensuring continuous key presses yield smooth, responsive movement.

Ensuring continuous key presses yield smooth, responsive movement.

Done

Task 2 is already implemented from earlier, so there's nothing left to add. Here is the code again.

`game/basket.py` (new method)

```
def move(self, direction, bounds_width):
    """Move by `direction` (-1 left, 0 none, +1 right) at the
basket's
    speed, keeping the whole basket inside [0, bounds_width]."""
    self.x += direction * self.speed
    half = self.width / 2
    self.x = max(half, min(bounds_width - half, self.x))
```

`game/game_engine.py` (`handle_input`)

Replace the old Left/Right branches and the `max(0, min(WIDTH, ...))` clamp with:

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return
    # Opposite keys held together cancel out (direction = 0).
    direction = int(keys_pressed[pygame.K_RIGHT]) -
int(keys_pressed[pygame.K_LEFT])
    self.basket.move(direction, WIDTH)
```

Why this covers the task

- **Boundaries:** the clamp uses the basket's half-width, so the whole basket stays on screen.
- **Smooth movement:** `main.py` polls `get_pressed()` every frame, so holding a key moves the basket 5 px per frame continuously.
- **Both keys held:** the movement cancels out instead of favoring one side.

Tests I ran headless:

- Holding Left for 500 frames leaves the left edge at 0.
- Holding Right for 500 frames leaves the right edge at 700.
- Pressing both keys at once leaves the basket where it is.

Ready for Task 3 (spawning) when you say.

10/11/2026, 5:58:58 PM

def handle_input(self, keys_pressed):
if self.game_over:
return
if keys_pressed[pygame.K_LEFT]:
self.basket.x -= self.basket.speed
if keys_pressed[pygame.K_RIGHT]:
self.basket.x += self.basket.speed
Boundary handling: only clamps against the screen edges, not
accounting for the basket's own width - it can hang half off
either side of the screen.
self.basket.x = max(0, min(WIDTH, self.basket.x))

what should i change this func to in task 2

10/11/2026, 6:02:04 PM

Replace it with this:

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return
    # Opposite keys held together cancel out (direction = 0).
    direction = int(keys_pressed[pygame.K_RIGHT]) -
int(keys_pressed[pygame.K_LEFT])
    self.basket.move(direction, WIDTH)
```

This calls `Basket.move`, so you also need to add it to `game/basket.py`, above `get_rect`:

```
def move(self, direction, bounds_width):
    self.x += direction * self.speed
```

```
    half = self.width / 2
    self.x = max(half, min(bounds_width - half, self.x))
```

If you'd rather keep everything in `handle_input` and skip `Basket.move`:

```
def handle_input(self, keys_pressed):
    if self.game_over:
        return
    if keys_pressed[pygame.K_LEFT]:
        self.basket.x -= self.basket.speed
    if keys_pressed[pygame.K_RIGHT]:
        self.basket.x += self.basket.speed
    half = self.basket.width / 2
    self.basket.x = max(half, min(WIDTH - half, self.basket.x))
```

The only real fix is the last line. Clamping to `[half, WIDTH - half]` keeps the whole basket on screen. The first version just moves that logic into `Basket` so Task 4's speed boost is easier to add.

10/11/2026, 6:02:07 PM

now do task 3 but give the entire code with the changed ones insted

10/11/2026, 6:03:27 PM

Implementing spawning changes across the game engine files.

- Implementing spawning changes across the game engine files.
- Implement Task 3 spawning, test headless
- Done

Task 3 is done and tested headless. It's only in my clone, nothing pushed. Only `game/game_engine.py` changed, and here is the entire file:

```
"""
GameEngine: owns the basket and all falling objects.

Starter version: basket movement and spawning both work at a basic
level (Tasks 2 and 3 ask you to improve them), there's no speed boost
yet (Task 4 builds it from scratch), and catch detection has two
known bugs (see game/collision.py and the catch-checking loop below)
that Task 1 asks you to fix.
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

MIN_SPAWN_INTERVAL_FRAMES = 30   # fastest gap between spawns (0.5s
at 60 FPS)
MAX_SPAWN_INTERVAL_FRAMES = 80   # slowest gap between spawns
```

```
MAX_OBJECTS_ON_SCREEN = 6        # hard cap so the game stays
playable
OBJECT_RADIUS = 14
SPAWN_MARGIN = OBJECT_RADIUS + 6 # keep objects fully inside the
screen
MIN_X_GAP = 80                   # new spawn must be this far from
the last one
MAX_SPAWN_TRIES = 10
MAX_MISSES = 5


class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
        self.frames_until_spawn = 0
        self.last_spawn_x = None
        self.score = 0
        self.misses = 0
        self.game_over = False

    def _pick_spawn_x(self):
        """Random x inside the playable area, at least MIN_X_GAP away
from
        the previous spawn (retries, then falls back to the far
side)."""
        lo, hi = SPAWN_MARGIN, WIDTH - SPAWN_MARGIN
        if self.last_spawn_x is None:
            return random.randint(lo, hi)
        for _ in range(MAX_SPAWN_TRIES):
            x = random.randint(lo, hi)
            if abs(x - self.last_spawn_x) >= MIN_X_GAP:
                return x
        # Fallback: guaranteed far from last spawn
        if self.last_spawn_x - lo >= hi - self.last_spawn_x:
```

```
            return random.randint(lo, max(lo, self.last_spawn_x -
MIN_X_GAP))
        return random.randint(min(hi, self.last_spawn_x + MIN_X_GAP),
hi)

    def _spawn_object(self):
        if len(self.objects) >= MAX_OBJECTS_ON_SCREEN:
            return
        x = self._pick_spawn_x()
        self.last_spawn_x = x
        self.objects.append(
            FallingObject(x=x, y=-OBJECT_RADIUS,
radius=OBJECT_RADIUS, speed=3)
        )

    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        # Opposite keys held together cancel out (direction = 0).
        direction = int(keys_pressed[pygame.K_RIGHT]) -
int(keys_pressed[pygame.K_LEFT])
        self.basket.move(direction, WIDTH)

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self.__init__()

    def update(self):
        if self.game_over:
            return

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_object()
            self.frames_until_spawn = random.randint(
```

```
                MIN_SPAWN_INTERVAL_FRAMES, MAX_SPAWN_INTERVAL_FRAMES
            )

        for obj in self.objects:
            obj.update()

        basket_rect = self.basket.get_rect()
        # Split into caught/remaining in one pass instead of removing
from
        # the list we're iterating over (which skipped neighbours).
        remaining = []
        for obj in self.objects:
            if is_caught(basket_rect, obj):
                self.score += 1
            else:
                remaining.append(obj)
        self.objects = remaining

        missed = [o for o in self.objects if
o.is_past_bottom(HEIGHT)]
        if missed:
            self.objects = [o for o in self.objects if not
o.is_past_bottom(HEIGHT)]
            self.misses += len(missed)
            if self.misses >= MAX_MISSES:
                self.game_over = True

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.basket, self.objects)
        renderer.draw_text(surface, font, f"Score: {self.score}",
(10, 10))
        renderer.draw_text(surface, font, f"Misses:
{self.misses}/{MAX_MISSES}", (10, 36))
```

```
        if self.game_over:
            renderer.draw_banner(surface, font, f"Game Over! Final
score: {self.score}. Press R to restart.")
```

This file also includes the Task 1 catch-loop fix and the Task 2 `handle_input` change. It still needs `Basket.move` in `game/basket.py` and the fixed `is_caught` in `game/collision.py` from the earlier tasks.

What changed for Task 3

- **Varied timing:** the gap between spawns is random, 30 to 80 frames, instead of a fixed 50.
- **Playable area:** x is picked within `SPAWN_MARGIN` (20 px) of each edge, so objects never spawn outside the screen.
- **No repeated spots:** a new spawn must be at least 80 px away from the previous one. It retries up to 10 times, then falls back to the far side of the screen.
- **Cap:** `_spawn_object` does nothing when 6 objects are already on screen.

Test results over 20,000 frames

- x stayed within 21 to 680, against an allowed range of 20 to 680.
- The smallest x difference between consecutive spawns was 80, which meets the minimum gap.
- Spawn intervals ranged from 30 to 80 frames.
- Forcing 20 spawns produced only 6 objects, so the cap holds.

The docstring at the top of the file still describes the starter version. Update it if you want it accurate.

Task 4 (speed boost) is next when you're ready.

10/11/2026, 6:03:57 PM

now task 4

10/11/2026, 6:04:14 PM

Debugging why cooldown behavior fails after a missed object.

- Updating basket mechanics and game engine for new feature.
- Implement Task 4 boost, test headless
- Debugging why cooldown behavior fails after a missed object.
- Retest cooldown with game-over avoided
- Done

Task 4 is done and tested headless. It's only in my clone, nothing pushed. Three files changed, and here they are in full.

`game/basket.py`

```
"""
Basket: the player-controlled catcher at the bottom of the screen.
"""

import pygame

BOOST_DURATION_FRAMES = 180    # 3s at 60 FPS
BOOST_COOLDOWN_FRAMES = 300    # 5s before it can be used again
BOOST_MULTIPLIER = 1.8


class Basket:
    def __init__(self, x, y, width=90, height=24, speed=5):
        self.x = x
        self.y = y
        self.width = width
        self.height = height
        self.speed = speed
        self.boosted_frames = 0
```

```
        self.cooldown_frames = 0
        self.normal_speed = speed
        self.boost_speed = speed * BOOST_MULTIPLIER

    @property
    def is_boosted(self):
        return self.boosted_frames > 0

    def activate_boost(self):
        """Start a boost if one isn't already running or cooling
down."""
        if self.is_boosted or self.cooldown_frames > 0:
            return False
        self.boosted_frames = BOOST_DURATION_FRAMES
        self.speed = self.boost_speed
        return True

    def update_boost(self):
        """Call once per frame: tick the boost down, restore speed on
expiry."""
        if self.boosted_frames > 0:
            self.boosted_frames -= 1
            if self.boosted_frames == 0:
                self.speed = self.normal_speed
                self.cooldown_frames = BOOST_COOLDOWN_FRAMES
        elif self.cooldown_frames > 0:
            self.cooldown_frames -= 1

    def move(self, direction, bounds_width):
        """Move by `direction` (-1 left, 0 none, +1 right) at the
basket's
        speed, keeping the whole basket inside [0, bounds_width]."""
        self.x += direction * self.speed
        half = self.width / 2
        self.x = max(half, min(bounds_width - half, self.x))
```

```
    def get_rect(self):
        return pygame.Rect(
            int(self.x - self.width / 2), int(self.y - self.height /
2),
            self.width, self.height,
        )
```

game/renderer.py

```
"""
renderer: all pygame drawing lives here, kept separate from game
logic.
"""

import pygame

WIDTH, HEIGHT = 700, 500
WINDOW_SIZE = (WIDTH, HEIGHT)

COLOR_BG = (25, 30, 45)
COLOR_BASKET = (150, 110, 70)
COLOR_BASKET_BOOST = (255, 200, 60)
COLOR_TEXT = (255, 255, 255)
COLOR_BOOST_BAR = (255, 200, 60)


def draw_scene(surface, basket, objects):
    surface.fill(COLOR_BG)
    for obj in objects:
        pygame.draw.circle(surface, obj.color, (int(obj.x),
int(obj.y)), obj.radius)
    color = COLOR_BASKET_BOOST if basket.is_boosted else COLOR_BASKET
    pygame.draw.rect(surface, color, basket.get_rect(),
border_radius=6)
```

```
def draw_text(surface, font, text, pos, color=COLOR_TEXT):
    surface.blit(font.render(text, True, color), pos)


def draw_banner(surface, font, text):
    surf = font.render(text, True, (255, 220, 80))
    rect = surf.get_rect(center=(surface.get_width() // 2,
surface.get_height() // 2))
    surface.blit(surf, rect)


def draw_boost_hud(surface, font, basket, pos, bar_width=120):
    """Boost status: active timer + bar, cooldown countdown, or ready
hint."""
    from game.basket import BOOST_DURATION_FRAMES
    x, y = pos
    if basket.is_boosted:
        secs = basket.boosted_frames / 60
        draw_text(surface, font, f"BOOST! {secs:.1f}s", pos,
COLOR_BOOST_BAR)
        frac = basket.boosted_frames / BOOST_DURATION_FRAMES
        pygame.draw.rect(surface, COLOR_BOOST_BAR, (x, y + 26,
int(bar_width * frac), 6))
    elif basket.cooldown_frames > 0:
        draw_text(surface, font, f"Boost cooling:
{basket.cooldown_frames / 60:.1f}s", pos, (150, 150, 150))
    else:
        draw_text(surface, font, "Boost ready [SPACE]", pos, (120,
220, 120))
```

game/game_engine.py

```
"""
GameEngine: owns the basket and all falling objects.
```

```
Handles input, controlled spawning, catch detection, misses, and the
temporary basket speed boost (SPACE).
"""

import random
import pygame

from game.basket import Basket
from game.falling_object import FallingObject
from game.collision import is_caught
from game.renderer import WIDTH, HEIGHT

MIN_SPAWN_INTERVAL_FRAMES = 30   # fastest gap between spawns (0.5s
at 60 FPS)
MAX_SPAWN_INTERVAL_FRAMES = 80   # slowest gap between spawns
MAX_OBJECTS_ON_SCREEN = 6        # hard cap so the game stays
playable
OBJECT_RADIUS = 14
SPAWN_MARGIN = OBJECT_RADIUS + 6 # keep objects fully inside the
screen
MIN_X_GAP = 80                   # new spawn must be this far from
the last one
MAX_SPAWN_TRIES = 10
MAX_MISSES = 5


class GameEngine:
    def __init__(self):
        self.basket = Basket(x=WIDTH / 2, y=HEIGHT - 30)
        self.objects = []
        self.frames_until_spawn = 0
        self.last_spawn_x = None
        self.score = 0
        self.misses = 0
        self.game_over = False
```

```
    def _pick_spawn_x(self):
        """Random x inside the playable area, at least MIN_X_GAP away
from
        the previous spawn (retries, then falls back to the far
side)."""
        lo, hi = SPAWN_MARGIN, WIDTH - SPAWN_MARGIN
        if self.last_spawn_x is None:
            return random.randint(lo, hi)
        for _ in range(MAX_SPAWN_TRIES):
            x = random.randint(lo, hi)
            if abs(x - self.last_spawn_x) >= MIN_X_GAP:
                return x
        # Fallback: guaranteed far from last spawn
        if self.last_spawn_x - lo >= hi - self.last_spawn_x:
            return random.randint(lo, max(lo, self.last_spawn_x -
MIN_X_GAP))
        return random.randint(min(hi, self.last_spawn_x + MIN_X_GAP),
hi)

    def _spawn_object(self):
        if len(self.objects) >= MAX_OBJECTS_ON_SCREEN:
            return
        x = self._pick_spawn_x()
        self.last_spawn_x = x
        self.objects.append(
            FallingObject(x=x, y=-OBJECT_RADIUS,
radius=OBJECT_RADIUS, speed=3)
        )

    def handle_input(self, keys_pressed):
        if self.game_over:
            return
        # Opposite keys held together cancel out (direction = 0).
        direction = int(keys_pressed[pygame.K_RIGHT]) -
```

```
int(keys_pressed[pygame.K_LEFT])
        self.basket.move(direction, WIDTH)

    def handle_keydown(self, key):
        if self.game_over and key == pygame.K_r:
            self.__init__()
        elif not self.game_over and key == pygame.K_SPACE:
            self.basket.activate_boost()

    def update(self):
        if self.game_over:
            return

        self.basket.update_boost()

        self.frames_until_spawn -= 1
        if self.frames_until_spawn <= 0:
            self._spawn_object()
            self.frames_until_spawn = random.randint(
                MIN_SPAWN_INTERVAL_FRAMES, MAX_SPAWN_INTERVAL_FRAMES
            )

        for obj in self.objects:
            obj.update()

        basket_rect = self.basket.get_rect()
        # Split into caught/remaining in one pass instead of removing
from
        # the list we're iterating over (which skipped neighbours).
        remaining = []
        for obj in self.objects:
            if is_caught(basket_rect, obj):
                self.score += 1
            else:
                remaining.append(obj)
```

```
        self.objects = remaining

        missed = [o for o in self.objects if
o.is_past_bottom(HEIGHT)]
        if missed:
            self.objects = [o for o in self.objects if not
o.is_past_bottom(HEIGHT)]
            self.misses += len(missed)
            if self.misses >= MAX_MISSES:
                self.game_over = True

    def draw(self, surface, font):
        from game import renderer
        renderer.draw_scene(surface, self.basket, self.objects)
        renderer.draw_text(surface, font, f"Score: {self.score}",
(10, 10))
        renderer.draw_text(surface, font, f"Misses:
{self.misses}/{MAX_MISSES}", (10, 36))
        renderer.draw_boost_hud(surface, font, self.basket, (10, 62))

        if self.game_over:
            renderer.draw_banner(surface, font, f"Game Over! Final
score: {self.score}. Press R to restart.")
```

How the boost works

- **Activate:** press SPACE. Speed goes from 5 to 9 px/frame for 3 seconds.
- **Indicator:** the basket turns gold and the HUD shows a "BOOST!" countdown with a shrinking bar.
- **Expiry:** speed returns to normal automatically.
- **Cooldown:** a 5-second cooldown follows, and the HUD shows it. Without it the player could hold the boost permanently. If you don't want that, set `BOOST_COOLDOWN_FRAMES = 0`.
- **Ignored presses:** SPACE does nothing while a boost is active, during cooldown, or after game over.

Tests

- Boost speed was 9 and a single move covered 9 px.
- Pressing SPACE again during a boost was ignored.
- After 180 frames the speed was back to 5 and the 300-frame cooldown had started.
- SPACE was blocked during cooldown and worked again once it finished.
- All three HUD states (active, cooling, ready) draw without errors.

One test I wrote failed at first, but it was my test's fault. Missed objects triggered game over mid-test, which froze the cooldown timer. With misses reset, it passes.

All four tasks are now done in the clone. I haven't pushed anything or run it with a real window. I'd run `python main.py` yourself for the before/after videos.

10/11/2026, 6:04:54 PM